

Per-Job Monitoring Data Collection with LDMS

LDMS User Group Meeting
Feb 9, 2026

Nichamon Naksinehaboon
nichamon@ogc.us
Open Grid Computing (OGC)

Recap from Last Meeting

What we covered:

- Multi-tenant environments: monitoring multiple concurrent jobs per node
- Tenant definitions as a combination of job attributes
- Tenant UUID tagging approach
- per-node metrics vs per-job metrics
- Per-job requires different collection approaches

Recap from Last Meeting (cont.)

- **Per-node metrics: (What LDMS collects today)**
 - Example: meminfo, vmstat, procstat
 - Measures: “Node has 80% CPU in use and 8GB free memory”
- **Per-job metrics: (What we don’t collect yet)**
 - Resource consumption by individual jobs
 - Measure: “Job 123 used 45% CPU, consumed 2 GB memory”

These are fundamentally different types of data

Per-Job Metrics Collection

- We will cover what's technically collectible
- Need your input on:
 - Which metrics matter most
 - Feasibility in your environments
 - Priority

Today's Coverage – Per-job Metrics

- Namespace Isolation Impact
- Metrics covered today
 - CPU, Memory, IO, Network
- Data sources
 - Cgroup, /proc/<pid>, eBPF
- Need your input on:
 - Which metrics matter most
 - Feasibility in your environments
 - Priority

Prerequisites & Configuration

Slurm cgroup.conf Requirements

Required controllers:

```
ConstrainCores=yes      # cpu.stat, cpu.pressure
ConstrainRAMSpace=yes    # memory.stat, memory.pressure
ConstrainSwapSpace=yes   # memory.swap.*
ConstrainDevices=yes     # Enables device control
```

Without these: No per-job accounting data

IO Accounting: Two-Step Setup

Step 1: Parent level (one-time)

```
echo "+io" >  
sys/fs/cgroup/system.slice/*_slurmstepd.scope/cgroup.subtree_control
```

→ Enables job-level io.stat

Step 2: Per-job (prolog script)

```
echo "+io" >  
/sys/fs/cgroup/system.slice/*_slurmstd.scope/job_${SLURM_JOB_ID}/cgroup.subtree_control
```

→ Enables step/task io.stat

Without step 2: Only job-level IO available

cgroup Hierarchy Structure

Slurm creates:

```
/sys/fs/cgroup/system.slice/*_slurmstepd.scope/  
  job_<jobid>/          ← always exists  
    step_<stepid>/      ← requires TaskPlugin=task/cgroup  
      task_<taskid>/    ← requires TaskPlugin=task/cgroup
```

Granularity = your TaskPlugin configuration

Namespace Isolation

Namespace Isolation Overview

Three isolation types affect visibility:

- PID namespace: Same data, different PID numbers
- Mount namespace: Isolates filesystem view (cgroups still accessible if mounted)
- Network namespace: Creates isolated network view
→ This enables per-job network stats

Isolation Creates Different Observability

Key concept:

- Same job
- Same processes
- Different namespace setup

→ Different observability

PID Namespace: PIDs Change

Host

- PIDs: 5001, 5002
- Network: host network

Job Environment

- PIDs: 1, 2
- Network: host network

Different IDs, same data

/proc/5001/stat (on host) = /proc/1/stat (in job env)

Network Namespace

Host

- eth0, ib0
- Host-wide counters

Job Environment

- eth0.123
- Isolated network

`/proc/<pid>/net/dev` shows different interfaces
This enables per-job network monitoring

The three Data Sources

cgroup v2:

- Job, step, task granularity

/proc/<pid>: Depends on namespace

- Process granularity → Need aggregation to obtain per-job

eBPF: Kernel level (future)

CPU Metrics

What CPU metrics can we collect per-job?

CPU from cgroup v2

Files: cpu.stat, cpu.pressure

What's available:

- Usage time (total, user, system)
- Throttling events and time
- Pressure stall information (PSI)

Granularity: Job / Step / Task

cgroup v2 – Usage Metrics (cpu.stat)

Key metrics:

- `usage_usec` - Total CPU time (microseconds)
- `user_usec` - Userspace CPU time
- `system_usec` - Kernel CPU time
- `nr_throttled` - Times throttled (Available when CPU controller enabled)
- `throttled_usec` - Total time throttled (Available when CPU controller enabled)

How to interpret:

- Delta between reads = CPU consumption
- `nr_throttled` > 0 = hitting CPU limits
- Compare to allocated cores for utilization %

cgroup v2 – CPU Pressure (cpu.pressure)

PSI - Pressure Stall Information

```
some avg10=0.00 avg60=0.00 avg300=0.00 total=2481  
full avg10=0.00 avg60=0.00 avg300=0.00 total=801
```

What this tells you:

- some = At least one task waiting for CPU
- full = ALL tasks waiting for CPU
- Moving averages: 10s, 60s, 300s windows

CPU from /proc/<pid>/stat

What's available:

- CPU time breakdown
 - utime - CPU time in user mode (per process)
 - stime - CPU time in kernel mode (per process)
 - cutime, cstime – Child process times
- Scheduling info: Last core executed on, priority, scheduling policy (NORMAL, FIFO), Thread count

CPU – cgroup v2 vs /proc/<pid>/stat Comparison

Prefer cgroup cpu.stat for per-job accounting:

- Job/step/task aggregation (with TaskPlugin)
- Throttling + CPU pressure stats
- No PID enumeration required

Use /proc/<pid>/*/stat when:

- CPU controller not enabled
- TaskPlugin not configured
- Need scheduling info (priority, policy, CPU affinity)
- Thread-level detail (/proc/<pid>/tasks/<tid>/stat)

Per-Core CPU - The Limitation

Neither cgroup v2 nor `/proc/<pid>` provides per-core time per job

What's missing:

- cgroup v1 had `cpuacct.usage_percpu` (per-core breakdown), removed in cgroup v2
- `/proc/<pid>/stat` shows which core (processor field), NOT time per core

What's available:

- System-wide per-core usage: `/proc/stat` (all jobs combined)
- Which core process last ran on: `/proc/<pid>/stat`
- Total job CPU time: cgroup `cpu.stat` (all cores combined)

Implication: Cannot determine "Job X used Y% of core 3"

Per-Core Efficiency Question

Scenario 1: Exclusive cores (ConstrainCores=yes)

- Job allocated specific cores (from `cpuset.cpus`)
- Can calculate: $\text{job_cpu_time} / (\text{allocated_cores} \times \text{interval}) = \text{aggregate efficiency}$
- But: Cannot break down which allocated core had what efficiency
- Get: "Job used 80% of its 4 allocated cores"
- Don't get: "Job used core 0 at 90%, core 1 at 70%"

Scenario 2: Shared cores

- Multiple jobs share same physical cores
- Linux scheduler time-slices between jobs
- Cannot attribute per-core usage to specific jobs
- System-wide `/proc/stat` shows core usage from all jobs combined

Bottom line: True per-core per-job breakdown not available from standard kernel interfaces

Memory Metrics

What memory metrics can we collect per-job?

Memory from cgroup v2

Selected Files: memory.stat, memory.events, memory.pressure

What's available:

- Usage breakdown (current, peak, anon, file, shmem, swap)
- OOM events and limit hits
- Memory contention (PSI)

Granularity: Job / Step / Task

cgroup v2 – Memory Usage from memory.stat

Key metrics:

- anon <bytes> # Anonymous memory (heap, stack)
- file <bytes> # File-backed memory (page cache, mapped)
- shmem <bytes> # Shared memory
- file_mapped <bytes> # Memory-mapped files

Critical for HPC:

- kernel memory overhead (slab, pagetables, kernel stacks)
- Current usage + peak tracking
- Swap details

Memory Events & Pressure

File: `memory.events`

- low 12 # Times below low threshold
- high 34 # Times above high threshold
- max 5 # Times hit memory.max limit
- oom 2 # Out-of-memory events
- oom_kill 1 # Processes killed by OOM

File: `memory.pressure`

some avg10=8.2 avg60=6.5 avg300=4.1 total=456789

full avg10=0.5 avg60=0.2 avg300=0.1 total=12345

Diagnostic value:

- full, then oom_kill → Pressure predicts kills
- high events → Approaching limits

Use case: Early warning for job slowdown before OOM

Memory from /proc/<pid>/status

What's available:

- Virtual memory: VmSize, VmPeak
- Per-segment breakdown: VmData (heap), VmStk (stack), VmExe (executable), VmLib (shared libraries)
- RSS breakdown: RssAnon, RssFile, RssShmem
- Per-process view (requires aggregation for per-job)

Memory - cgroup v2 vs /proc/<pid>/status

Prefer cgroup memory.stat:

- Job/step/task aggregation (with TaskPlugin)
- Includes kernel overhead (missing from /proc sum)
- Shared memory counted once (not duplicated per-process)
- Pressure diagnostics included

Use /proc/<pid>/status for:

- Per-thread detail (/proc/<pid>/task/<tid>/status)
- Virtual memory analysis

IO Metrics

What IO metrics can we collect per-job?

IO from cgroup v2

Files: io.stat, io.pressure

What's available:

- Block device IO statistics
- Per-device breakdown
- Pressure information (IO wait)

Granularity: Typically job-level (step/task needs prolog to enable +io)

cgroup v2 – IO Statistics from io.stat

Per-device metrics:

```
17# cat io.stat  
253:0 rbytes=0 wbytes=8192 rios=0 wios=1 dbytes=0 dios=0
```

Key metrics:

- Device major:minor ID (8:0, 8:16, etc.)
- rbytes/wbytes - Bytes read/written
- rios/wios - Number of read/write operations
- Discard operations

What it captures: Block device level only (not filesystem level)

cgroup v2 – IO Pressure from io.pressure

```
17# cat io.pressure
some avg10=0.00 avg60=0.00 avg300=0.00 total=0
full avg10=0.00 avg60=0.00 avg300=0.00 total=0
```

What it tells you:

- some: At least one task waiting for IO
- full: All tasks waiting for IO
- Indicates IO contention/bottlenecks

Limitations of cgroup IO

- Block devices only (not filesystem-level)
- No network filesystem IO (NFS, Lustre, GPFS, BeeGFS)
- No cached vs uncached distinction
- Requires prolog for step/task levels

IO from /proc/<pid>/io

What's available:

- Logical IO (includes cached):
 - rchar, wchar (characters read/written including cache)
 - syscr, syscw (read/write syscall counts)
- Physical IO (actual storage):
 - read_bytes, write_bytes (actual disk IO)
- cancelled_write_bytes (writes that were cancelled)

IO – cgroup v2 vs /proc/<pid>/io

Prefer cgroup io.stat:

- Job/step/task aggregation (with TaskPlugin + prolog)
- Per-device breakdown
- Physical IO (what hit storage)

Use /proc/<pid>/io:

- Cached vs physical IO distinction (rchar/wchar)
- When TaskPlugin not configured

Limitation: Neither captures network filesystem IO

Network metrics

What network metrics can we collect per-job?

The Network Challenge

Critical gap:

- cgroup v2 has NO network controller

Why this matters:

- Cannot get network bytes/packets from cgroup filesystem

Network from /proc - The Problem

File exists: /proc/<pid>/net/dev

Without network namespace isolation:

```
# /proc/<pid>/net/dev shows:  
eth0: <host-wide counters>  
ib0: <host-wide counters>
```

- Shows ALL traffic on host interfaces
- NOT per-process
- NOT per-job
- Same view for every PID

Result: Not useful for per-job monitoring

Why: All processes on host see the same host-wide network stats

Network from /proc - A Solution

WITH **network namespace isolation**:

Job (in isolated network namespace) sees:

```
# /proc/<pid>/net/dev shows:  
eth0.veth123: <job-specific counters>
```

- Only job's assigned virtual interface
- Only job's traffic
- Isolated from other jobs
- Containerized environment makes this easy

Result: Job-specific network statistics

Network namespace required for per-job network from /proc

Network Collection Approach

Requirements:

- Network namespace isolation per job
- Per-job Idmsd running inside job namespace
- Or host Idmsd with nsenter capability

Note: All PIDs in same namespace see same interface, so could read once

eBPF for Network - Future Work

What eBPF solves:

- Network accounting without namespace requirement
- Zero-loss capture (no sampling gaps)
- Works at kernel level across all namespaces

Benefits:

- Every packet attributed to correct job
- Works regardless of namespace configuration

How eBPF Work

BPF programs attach to kernel network functions:

- tcp_sendmsg, tcp_recvmsg (TCP)
- udp_sendmsg, udp_recvmsg (UDP)
- socket operations

On each call:

- Get current PID
- Look up job - PID map (E.g., cgroup membership)
- Increment per-job counter in kernel memory (atomic)

Idmsd reads BPF map periodically for accumulated totals

eBPF Requirements

Kernel requirements:

- Linux 4.18+ (basic BPF)
- Linux 5.4+ recommended (BPF CO-RE)
- BTF (BPF Type Format) enabled
- libbpf library

Development complexity:

- BPF programs in restricted C
- Requires compilation toolchain (clang, llvm)
- Kernel version compatibility matrix
- More complex than reading files

Timeline: Future enhancement

Current approach: /proc with network namespace isolation

Summary

Summary by Metric Type

CPU:

- cgroup (preferred): usage, throttling, pressure
- /proc (fallback): per-process time
- Limitation: No per-core per-job

Memory:

- cgroup (preferred): current, peak, pressure, events
- /proc (detail): per-process/per-thread breakdown

IO:

- cgroup: block device level, pressure
- /proc: includes cached IO
- Limitation: No network filesystems

Network:

- /proc inside job environment: requires network namespace
- eBPF (future): without namespace requirement

Data Visibility Comparison

- With PID Namespace isolation

Data Source	Host Context	Job Context	Identical?
/sys/fs/cgroup/../../job_<id>/	✓ Accessible	✓ Accessible	✓ Same data
/proc/<PID>/stat	✓ /proc/5001/stat	✓ /proc/1/stat	same data, different PID
/proc/<PID>/status	✓ /proc/5001/status	✓ /proc/1/status	same data, different PID
/proc/<PID>/io	✓ /proc/5001/io	✓ /proc/1/io	same data, different PID
/proc/<PID>/net/dev (no net ns)	Host-wide stats	Host-wide stats	Same (not job-specific)
/proc/<PID>/net/dev (with net ns)	Host-wide stats	Job interfaces only	Different

Key Limitations

CPU:

- No per-core per-job breakdown
- PSI doesn't observe busy-wait spinlocks or RDMA

Memory:

- (No major limitations from cgroup/proc)

IO:

- Block devices only (no NFS, Lustre, GPFS, BeeGFS)
- No per-file or per-path breakdown
- io.stat requires prolog for step/task levels

Network:

- cgroup v2 has no network controller
- /proc requires network namespace isolation for per-job stats
- Without namespace: only host-wide stats available
- Doesn't observe kernel-bypass transports (RDMA, GPU Direct)

Discussion

Cgroup Requirements

This approach requires:

- Cgroup v2
- ConstrainCores=yes, ConstrainRAMSpace=yes, ConstrainDevices=yes
- TaskPlugin=task/cgroup (for step/task breakdown)

Your environment:

- Can enable these controllers?
- Already on cgroup v2?
- Policies blocking cgroup configuration?

Show of hands: Any cgroup blockers?

Feedback during the meeting: If cgroup is used, it's cgroup v2, so no need to support cgroup v1.

Metric Priorities

Granularity:

- Job-level metrics sufficient for your use cases?
- Or need step-level and task-level breakdown?

→ Impacts: TaskPlugin configuration, prolog complexity

Feedback during the meeting: job-level has higher priority and currently job-level is sufficient granularity.
Step and task/process has lower priority.

Pressure Stall Information (PSI):

- Interest in *.pressure metrics (CPU, memory, IO contention)?
- Your use case:
 - Early warning before OOM/throttling
 - Just want usage tracking
 - Not sure yet

Show of hands: Who wants PSI metrics as high priority?

Feedback during the meeting: There was no explicit interests to have the PSI metrics in the first roll out.

Environment & Deployment - Your Input Needed

Namespace isolation status:

- Using today? Which types: PID / mount / network
- Planning to adopt for multi-tenant environments?
- Why this matters: Network namespace enables per-job network stats

LDMSD deployment model:

- Can you run ldmsd inside job environment?
- Can job ldmsd connect to host ldmsd/aggregator?
- Any privilege restrictions for monitoring?
- Why this matters: Determines collection architecture
 - Host-based collection (read cgroup from host)
 - Job-based collection (ldmsd inside namespace)

Any site policies blocking namespace adoption?

GPU

Vendor/model:

- NVIDIA, AMD, Intel?

Sharing model:

- Exclusive allocation per job
- MPS (Multi-Process Service)
- MIG (Multi-Instance GPU)
- Time-slicing

Metrics needed:

- GPU utilization
- GPU memory usage
- Temperature
- Power consumption

Current monitoring:

- Do you already collect GPU metrics?
- What tools/methods currently used?
 - If using Idmsd, what sampler plugins?

Q&A:

Conclusion from the meeting:

- Per-job CPU, Memory, and IO metrics from the cgroup v2 has no blocker, so `cpu.stat`, `memory.stat`, and `io.stat` will be the data source
- Per-job network metrics will be from `/proc/<pid>/net/dev` when network namespace isolation is used

Feedback

- Different job schedulers and different job scheduler configuration create different cgroup directory structure, so assuming a specific structure will prevent portability
 - A solution: make cgroup directory structure configurable
- Flux, job scheduler, always creates cgroup for each job

Thank you!

Contact nichamon@ogc.us for questions or feedback